pkshatech / **GLuCoSE-base-ja-v2** like 21

Follow PKSHA Technology Inc. 30

Sentence Similarity | sentence-transformers | Safetensors | hpprc/emb | hpprc/mqa-ja | google-research-datasets/paws-x | Japanese | luke | feature-extraction | License: apache-2.0

Train | Deploy | Use this model

Model card | Files xet | Community 1

Downloads last month
6,913

Safetensors Model size 133M params | Tensor type F32 | Files info

Inference Providers NEW

Sentence Similarity

Source Sentence

Your sentence here...

Sentences to compare to

Your sentence here...

Your sentence here...

Add Sentence

Generate

View Code

Run 15,000+ Models Instantly

Inference Providers let you run inference on thousands of models served by our partners using a simple, unified, OpenAI-compatible serverless API (Learn more).

`pkshatech/GLuCoSE-base-ja-v2` is supported by the following Inference Providers:

HF Inference API

View API Code | Dismiss

Model tree for `pkshatech/GLuCoSE-base-ja-v2`

Base model `studio-ousia/luke-japanese-base-lite`

Finetuned `pkshatech/GLuCoSE-base-ja`

Finetuned (1) this model

Datasets used to train `pkshatech/GLuCoSE-base-ja-v2`

Space using `pkshatech/GLuCoSE-base-ja-v2` 1

GLuCoSE v2

This model is a general Japanese text embedding model, excelling in retrieval tasks. It can run on CPU and is designed to measure semantic similarity between sentences, as well as to function as a retrieval system for searching passages based on queries.

Key features:

- Specialized for retrieval tasks, it demonstrates the highest performance among similar size models in MIRACL and other tasks .
- Optimized for Japanese text processing
- Can run on CPU

During inference, the prefix "query: " or "passage: " is required. Please check the Usage section for details.

Model Description

The model is based on GLuCoSE and fine-tuned through distillation using several large-scale embedding models and multi-stage contrastive learning.

- **Maximum Sequence Length:** 512 tokens
- **Output Dimensionality:** 768 tokens
- **Similarity Function:** Cosine Similarity

Usage

Direct Usage (Sentence Transformers)

You can perform inference using SentenceTransformer with the following code:

```
from sentence_transformers import SentenceTransformer
import torch.nn.functional as F

# Download from the 🤗 Hub
model = SentenceTransformer("pkshatech/GLuCoSE-base-ja-v2")

# Each input text should start with "query: " or "passage: ".
# For tasks other than retrieval, you can simply use the "query: " pre
sentences = [
    'query: PKSHAはどんな会社ですか？',
    'passage: 研究開発したアルゴリズムを、多くの企業のソフトウエア・オペレーション
```

```
    'query: 日本で一番高い山は？',
    'passage: 富士山（ふじさん）は、標高3776.12 m、日本最高峰（剣ヶ峰）の独立峰て
]
embeddings = model.encode(sentences,convert_to_tensor=True)
print(embeddings.shape)
# [4, 768]

# Get the similarity scores for the embeddings
similarities = F.cosine_similarity(embeddings.unsqueeze(0), embeddings
print(similarities)
# [[1.0000, 0.6050, 0.4341, 0.5537],
# [0.6050, 1.0000, 0.5018, 0.6815],
# [0.4341, 0.5018, 1.0000, 0.7534],
# [0.5537, 0.6815, 0.7534, 1.0000]]
```

Direct Usage (Transformers)

You can perform inference using Transformers with the following code:

```
import torch.nn.functional as F
from torch import Tensor
from transformers import AutoTokenizer, AutoModel

def mean_pooling(last_hidden_states: Tensor,attention_mask: Tensor) ->
    emb = last_hidden_states * attention_mask.unsqueeze(-1)
    emb = emb.sum(dim=1) / attention_mask.sum(dim=1).unsqueeze(-1)
    return emb

# Download from the 🤗 Hub
tokenizer = AutoTokenizer.from_pretrained("pkshatech/GLuCoSE-base-ja-v2
model = AutoModel.from_pretrained("pkshatech/GLuCoSE-base-ja-v2")

# Each input text should start with "query: " or "passage: ".
# For tasks other than retrieval, you can simply use the "query: " pref
sentences = [
    'query: PKSHAはどんな会社ですか？',
    'passage: 研究開発したアルゴリズムを、多くの企業のソフトウエア・オペレーション
    'query: 日本で一番高い山は？',
    'passage: 富士山（ふじさん）は、標高3776.12 m、日本最高峰（剣ヶ峰）の独立峰て
]

# Tokenize the input texts
batch_dict = tokenizer(sentences, max_length=512, padding=True, truncat

outputs = model(**batch_dict)
embeddings = mean_pooling(outputs.last_hidden_state, batch_dict['atten
```

```
print(embeddings.shape)
# [4, 768]

# Get the similarity scores for the embeddings
similarities = F.cosine_similarity(embeddings.unsqueeze(0), embeddings
print(similarities)
# [[1.0000, 0.6050, 0.4341, 0.5537],
# [0.6050, 1.0000, 0.5018, 0.6815],
# [0.4341, 0.5018, 1.0000, 0.7534],
# [0.5537, 0.6815, 0.7534, 1.0000]]
```

Training Details

The fine-tuning of GLuCoSE v2 is carried out through the following steps:

Step 1: Ensemble distillation

- The embedded representation was distilled using E5-mistral, gte-Qwen2, and mE5-large as teacher models.

Step 2: Contrastive learning

- Triplets were created from JSNLI, MNLI, PAWS-X, JSeM and Mr.TyDi and used for training.
- This training aimed to improve the overall performance as a sentence embedding model.

Step 3: Search-specific contrastive learning

- In order to make the model more robust to the retrieval task, additional two-stage training with QA and retrieval task was conducted.
- In the first stage, the synthetic dataset auto-wiki-qa was used for training, while in the second stage, JQaRA, MQA, Japanese Wikipedia Human Retrieval, Mr.TyDi,MIRACL, Quiz Works and Quiz No Mori were used.

Benchmarks

Retrieval

Evaluated with MIRACL-ja, JQARA , JaCWIR and MLDR-ja.

Model	Size	MIRACL Recall@5	JQaRA nDCG@10	JaCWIR MAP@10	MLDR nDCG@10
intfloat/multilingual-e5-large	0.6B	89.2	55.4	**87.6**	29.8
cl-nagoya/ruri-large	0.3B	78.7	62.4	85.0	**37.5**
intfloat/multilingual-e5-base	0.3B	84.2	47.2	**85.3**	25.4
cl-nagoya/ruri-base	0.1B	74.3	58.1	84.6	**35.3**
pkshatech/GLuCoSE-base-ja	0.1B	53.3	30.8	68.6	25.2
GLuCoSE v2	0.1B	**85.5**	**60.6**	**85.3**	33.8

Note: Results for OpenAI small embeddings in JQARA and JaCWIR are quoted from the JQARA and JaCWIR.

JMTEB

Evaluated with JMTEB. The average score is macro-average.

Model	Size	Avg.	Retrieval	STS	Classification	Reranking	Cluste
OpenAI/text-embedding-3-small	-	69.18	66.39	79.46	73.06	92.92	51.
OpenAI/text-embedding-3-large	-	74.05	74.48	82.52	77.58	93.58	53.
intfloat/multilingual-e5-large	0.6B	70.90	70.98	79.70	72.89	92.96	51.
cl-nagoya/ruri-large	0.3B	73.31	73.02	83.13	77.43	92.99	51.
intfloat/multilingual-e5-base	0.3B	68.61	68.21	79.84	69.30	**92.85**	48.
cl-nagoya/ruri-base	0.1B	71.91	69.82	82.87	75.58	92.91	**54.**
pkshatech/GLuCoSE-base-ja	0.1B	67.29	59.02	78.71	**76.82**	91.90	49.
GLuCoSE v2	0.1B	**72.23**	**73.36**	**82.96**	74.21	93.01	48.

Note: Results for OpenAI embeddings and multilingual-e5 models are quoted from the JMTEB leaderboard. Results for ruri are quoted from the cl-nagoya/ruri-base model card.

Authors

Chihiro Yano, Mocho Go, Hideyuki Tachibana, Hiroto Takegawa, Yotaro Watanabe

License